004 -- Strategy Construction

Author: Wayne Kirk Schmidt
Email: wayne.kirk.schmidt@gmail.com

Purpose

`004_strategy.ipynb` is the fourth stage of the cryptocurrency statistical arbitrage research pipeline.

This stage converts the structured shock event data from Stage 3 into **evaluated trading candidates**. Rather than defining a fully deployable trading system, the goal here is to measure whether the observed lead-lag relationships represent a statistically meaningful and repeatable edge.

Inputs

From `output/001_download/`, `output/002_enrich/`, and `output/003_analysis/`:

File	Description
`PRICES.pkl`	Long-format daily OHLCV
`<COIN>.event_panel.pkl`	Per-asset event panels
`returns_full.pkl`	Synchronized daily return matrix
`z_scores.pkl`	Standardized return matrix
`rolling_sigma.pkl`	Rolling volatility estimates
`price_wide.pkl`	Wide-format close price matrix
`event_bitmap.pkl`	Multi-asset shock co-occurrence table
`sigma_event_matrix.pkl`	Per-asset event counts by sigma level

Outputs

Written to `output/004_strategy/`:

File	Description
`event_response_df.pkl`	Base event-response dataset (with returns vectors)
`event_response_expanded.pkl`	Expanded dataset with per-lag return columns
`trade_df.pkl`	Full mapped trade representation
`reduced_df.pkl`	High-quality subset using the conditioned signal
`daily_returns.pkl`	Aggregated daily return series

Analysis Goals

1. **Construct the event response dataset** -- for each shock event, record how every other asset performed over the subsequent observation window
2. **Measure directional consistency** -- quantify mean response and signal-to-noise at each lag
3. **Identify the signal** -- find the specific lag and condition that maximizes predictive value
4. **Build trade candidates** -- map the conditioned signal to a trade representation for backtesting

Pipeline Position

```
001_download  -> data acquisition
002_enrich    -> feature engineering
003_analysis  -> shock detection
004_strategy  <- you are here
    -
005_backtest  -> execution and performance evaluation
```

Design Principles

- **Deterministic** -- no recomputation of prior stages; all inputs are loaded from persisted artifacts
- **No forward-looking assumptions** -- all signal construction uses only information available at the time of the event
- **Separation from backtesting** -- this stage identifies the signal; Stage 5 evaluates it under realistic execution assumptions

1. Imports and Environment Setup

Standard library imports for data manipulation, numerical computation, file I/O, and visualization.

In [1]:
```
import pandas as pd
import numpy as np
import pickle
from datetime import datetime, UTC
import math
from pathlib import Path
import matplotlib.pyplot as plt
```

2. Environment Configuration

Defines the asset universe, pipeline parameters, and output directories for this stage.

Key parameter: `sigma_threshold = 3` -- events below this magnitude are not considered for signal construction.

In [2]:
```
startdate = "2023-01-01"
trading_days = 252
frequency = "1d"

universe = [
    "BTCUSDT",   # Bitcoin
    "ETHUSDT",   # Ethereum
    "BNBUSDT",   # Binance Coin
    "SOLUSDT",   # Solana
    "XRPUSDT",   # Ripple
    "ADAUSDT",   # Cardano
    "DOGEUSDT",  # Dogecoin
    "AVAXUSDT",  # Avalanche
    "LTCUSDT"    # Litecoin
]

execution_delay = [0, 1, 2, 3]
execution_cost_bps = [20, 30, 40]

stage_label = "004_strategy"

OUTPUT_ROOT = Path("../output")
OUTPUT_ROOT.mkdir(parents=True, exist_ok=True)

MANIFEST_FILE = OUTPUT_ROOT / "manifest.pkl"

DOWNLOAD_DIR = OUTPUT_ROOT / "001_download"
DOWNLOAD_DIR.mkdir(parents=True, exist_ok=True)

ENRICH_DIR = OUTPUT_ROOT / "002_enrich"
ENRICH_DIR.mkdir(parents=True, exist_ok=True)

ANALYSIS_DIR = OUTPUT_ROOT / "003_analysis"
ANALYSIS_DIR.mkdir(parents=True, exist_ok=True)

STRATEGY_DIR = OUTPUT_ROOT / "004_strategy"
STRATEGY_DIR.mkdir(parents=True, exist_ok=True)

inspection_window = 20

sigma_threshold = 3

observation_window_length = 10
observation_window = range(1, observation_window_length + 1)

holding_period = 1
```

3. Load or Initialize the Manifest

Loads the shared pipeline manifest and initializes this stage's entry if not already present.

In [3]:
```
if MANIFEST_FILE.exists():
    manifest = pd.read_pickle(MANIFEST_FILE)
else:
    manifest = {}

manifest.setdefault(stage_label, {})
```

Out[3]:
```
{}
```

4. Load Prior Stage Artifacts

Loads all required artifacts from Stages 1, 2, and 3. Shape diagnostics confirm each dataset loaded correctly before analysis begins.

In [4]:
```
# 001_download artifacts
prices = pd.read_pickle(DOWNLOAD_DIR / "PRICES.pkl")
event_panels = {}
for coin in universe:
    panel_file = DOWNLOAD_DIR / f"{coin}.event_panel.pkl"
    event_panels[coin] = pd.read_pickle(panel_file)

# 002_enrich artifacts
returns_full = pd.read_pickle(ENRICH_DIR / "returns_full.pkl")
z_scores = pd.read_pickle(ENRICH_DIR / "z_scores.pkl")
rolling_sigma = pd.read_pickle(ENRICH_DIR / "rolling_sigma.pkl")
price_wide = pd.read_pickle(ENRICH_DIR / "price_wide.pkl")

# 003_analysis artifacts
event_bitmap = pd.read_pickle(ANALYSIS_DIR / "event_bitmap.pkl")
extreme_z_scores = pd.read_pickle(ANALYSIS_DIR / "extreme_z_scores.pkl")
sigma_event_matrix = pd.read_pickle(ANALYSIS_DIR / "sigma_event_matrix.pkl")

print("prices:", prices.shape)
print("returns_full:", returns_full.shape)
print("z_scores:", z_scores.shape)
print("rolling_sigma:", rolling_sigma.shape)
print("price_wide:", price_wide.shape)
print("event_bitmap:", event_bitmap.shape)
print("extreme_z_scores:", extreme_z_scores.shape)
print("sigma_event_matrix:", sigma_event_matrix.shape)
print("event panels loaded:", list(event_panels.keys()))
```

```
prices: (10984, 5)
returns_full: (1047, 9)
z_scores: (1028, 9)
rolling_sigma: (1028, 9)
price_wide: (1242, 9)
event_bitmap: (19, 11)
extreme_z_scores: (9, 5)
sigma_event_matrix: (9, 5)
event panels loaded: ['BTCUSDT', 'ETHUSDT', 'BNBUSDT', 'SOLUSDT', 'XRPUSDT', 'ADAUSDT', 'DOGEUSDT', 'AVAXUSDT', 'LTCUSDT']
```

5. Reconstruct the Signed Sigma Classification Matrix

Builds a signed indicator matrix from the z-score data:

- `+1` where the z-score meets or exceeds the threshold (positive shock)
- `-1` where the z-score is at or below the negative threshold (negative shock)
- `0` otherwise

The assertion validates that the exact expected number of qualifying events is present -- a regression check against prior pipeline runs.

In [5]:
```
assert "sigma_event_matrix" in locals(), "sigma_event_matrix not loaded"

sigma_class = (z_scores > sigma_threshold).astype(int) - (z_scores < -sigma_threshold).astype(int)

print("max:", z_scores.max().max())
print("min:", z_scores.min().min())
print("event_count:", ((z_scores >= 3) | (z_scores <= -3)).sum().sum())

sigma_class = (z_scores >= sigma_threshold).astype(int) - (z_scores <= -sigma_threshold).astype(int)

assert sigma_class.shape == z_scores.shape
assert int((sigma_class != 0).sum().sum()) == 104

print("OK:", sigma_class.shape, "events:", int((sigma_class != 0).sum().sum()))
```

```
max: 4.281477768984525
min: -4.155916934268655
event_count: 104
OK: (1028, 9) events: 104
```

6. Review Signed Sigma Distribution

Displays the frequency distribution of signed sigma buckets across all assets and dates. This confirms the relative frequency of up-shock versus down-shock events and validates the classification matrix.

In [6]:
```
sigma_class = np.floor(z_scores).astype(int)

# count signed sigma values
sigma_counts = sigma_class.stack().value_counts().sort_index()

print(sigma_counts)
```

```
-5       4
-4      40
-3     187
-2     976
-1    3419
 0    3276
 1     998
 2     292
 3      58
 4       2
Name: count, dtype: int64
```

7. Event Breakdown by Sigma Level and Coin

Cross-tabulates event counts by signed sigma value and coin. This reveals which assets are most frequently the source of extreme shocks and whether the distribution is symmetric between positive and negative moves.

In [7]:
```
# count by signed sigma and coin
pd.set_option("display.width", 2000)
pd.set_option("display.max_columns", None)
stacked = sigma_class.stack()

sigma_coin_counts = (
    stacked
    .groupby([stacked, stacked.index.get_level_values(1)])
    .size()
    .unstack(fill_value=0)
    .sort_index()
)

extreme_mask = (sigma_coin_counts.index <= -3) | (sigma_coin_counts.index >= 3)

print(sigma_coin_counts)
```

```
coin  ADAUSDT  AVAXUSDT  BNBUSDT  BTCUSDT  DOGEUSDT  ETHUSDT  LTCUSDT  SOLUSDT  XRPUSDT
-5          0         0        0        1         0        1        0        0        2
-4          6         3        5        3         3        5        7        3        5
-3         16        19       28       24        21       23       20       18       18
-2        121       120       96      100       100      111      104      118      106
-1        390       376      355      375       405      360      382      374      402
 0        348       359      395      363       353      383      377      340      358
 1        107       112      106      122       107      110      106      134       94
 2         37        34       36       33        31       28       24       36       33
 3          2         5        7        7         8        6        8        5       10
 4          1         0        0        0         0        1        0        0        0
```

8. Isolate Extreme Events Per Coin

Applies the extreme mask (|sigma| ≥ 3) to the cross-tabulation, isolating the events of analytical interest. This is the set from which all trading signals will be derived.

In [8]:
```
extreme_sigma_coin_counts = sigma_coin_counts.loc[extreme_mask]

print(extreme_sigma_coin_counts)
```

```
coin  ADAUSDT  AVAXUSDT  BNBUSDT  BTCUSDT  DOGEUSDT  ETHUSDT  LTCUSDT  SOLUSDT  XRPUSDT
-5          0         0        0        1         0        1        0        0        2
-4          6         3        5        3         3        5        7        3        5
-3         16        19       28       24        21       23       20       18       18
 3          2         5        7        7         8        6        8        5       10
 4          1         0        0        0         0        1        0        0        0
```

9. Build the Long-Format Event Table

Converts the wide sigma classification matrix into a long-format table where each row is one (date, reference coin, sigma) tuple. Only extreme events (|sigma| ≥ 3) are retained. This is the base dataset for attaching forward return vectors.

In [9]:
```
leader_mask = (sigma_class >= 3) | (sigma_class <= -3)
leader_events = sigma_class.where(leader_mask)

# convert to long format
event_df = (
    leader_events
    .stack()
    .reset_index()
)

# standardize column names
event_df.columns = ["event_date", "reference_coin", "sigma"]

event_df = event_df.dropna(subset=["sigma"])
event_df["sigma"] = event_df["sigma"].astype(int)

# sort for sanity
event_df = event_df.sort_values(["event_date", "reference_coin"]).reset_index(drop=True)

# quick checks
print("event_df shape:", event_df.shape)
print(event_df.head())
```

```
event_df shape: (291, 3)
                 event_date reference_coin  sigma
0 2023-08-03 00:00:00+00:00        LTCUSDT     -3
1 2023-08-15 00:00:00+00:00       AVAXUSDT     -3
2 2023-08-15 00:00:00+00:00       DOGEUSDT     -3
3 2023-08-15 00:00:00+00:00        SOLUSDT     -3
4 2023-08-15 00:00:00+00:00        XRPUSDT     -3
```

10. Attach Forward Return Vectors

For each shock event (reference coin, date), extracts the forward return path of every other asset in the universe over the `observation_window_length` subsequent trading days.

Each row in the resulting dataset represents one (reference coin -> target coin, date) pair with a full vector of forward returns -- the raw material for lag analysis.

In [10]:
```
all_coins = price_wide.columns.tolist()

records = []

for _, row in event_df.iterrows():
    event_date = row["event_date"]
    ref_coin = row["reference_coin"]
    sigma = row["sigma"]

    # locate event index
    if event_date not in price_wide.index:
        continue

    idx = price_wide.index.get_loc(event_date)

    for target_coin in all_coins:
        if target_coin == ref_coin:
            continue

        # ensure full forward window exists
        if idx + observation_window_length >= len(price_wide):
            continue

        # extract forward prices
        price_series = price_wide[target_coin].iloc[idx : idx + observation_window_length + 1]

        # convert to returns
        returns = price_series.pct_change().iloc[1:].values

        if len(returns) != observation_window_length:
            continue

        records.append({
            "event_date": event_date,
            "reference_coin": ref_coin,
            "target_coin": target_coin,
            "sigma": sigma,
            "returns_vector": returns
        })

event_response_df = pd.DataFrame(records)

print("event_response_df shape:", event_response_df.shape)
print(event_response_df.head())
```

```
EVENT_RESPONSE_RAW_FILE = STRATEGY_DIR / "event_response_df.pkl"
event_response_df.to_pickle(EVENT_RESPONSE_RAW_FILE)
manifest[stage_label]["event_response_df"] = str(EVENT_RESPONSE_RAW_FILE)
print("saved:", EVENT_RESPONSE_RAW_FILE)
print("shape:", event_response_df.shape)
```

```
event_response_df shape: (2320, 5)
                 event_date reference_coin target_coin  sigma                                    returns_vector
0 2023-08-03 00:00:00+00:00        LTCUSDT     ADAUSDT     -3  [0.004098360655737654, -0.0010204081632652073,...
1 2023-08-03 00:00:00+00:00        LTCUSDT    AVAXUSDT     -3  [-0.003212851405622441, 0.00241740531829171, 0...
2 2023-08-03 00:00:00+00:00        LTCUSDT     BNBUSDT     -3  [0.0016625103906899863, 0.007883817427385864, ...
3 2023-08-03 00:00:00+00:00        LTCUSDT     BTCUSDT     -3  [-0.00256893391640145, -0.0018550830546302244,...
4 2023-08-03 00:00:00+00:00        LTCUSDT    DOGEUSDT     -3  [-0.0017643865363733413, 0.028687967369136702,...
saved: ..\output\004_strategy\event_response_df.pkl
shape: (2320, 5)
```

11. Expand Returns Vectors into Lag Columns

Flattens the returns vector into individual columns (`lag_1` through `lag_10`) for vectorized analysis. This structure allows direct groupby and filtering operations across lag positions without unpacking arrays row by row.

In [11]:
```
lags = 10

# expand returns_vector into columns
returns_expanded = pd.DataFrame(
    event_response_df["returns_vector"].tolist(),
    columns=[f"lag_{i+1}" for i in range(lags)]
)

# combine back
event_response_expanded = pd.concat(
    [event_response_df.drop(columns=["returns_vector"]), returns_expanded],
    axis=1
)

print(event_response_expanded.shape)
print(event_response_expanded.head())

EVENT_RESPONSE_EXPANDED_FILE = STRATEGY_DIR / "event_response_expanded.pkl"
event_response_expanded.to_pickle(EVENT_RESPONSE_EXPANDED_FILE)
manifest[stage_label]["event_response_expanded"] = str(EVENT_RESPONSE_EXPANDED_FILE)
print("saved:", EVENT_RESPONSE_EXPANDED_FILE)
print("shape:", event_response_expanded.shape)
```

```
(2320, 14)
                 event_date reference_coin target_coin  sigma     lag_1     lag_2     lag_3     lag_4     lag_5     lag_6     lag_7     lag_8     lag_9    lag_10
0 2023-08-03 00:00:00+00:00        LTCUSDT     ADAUSDT     -3  0.004098 -0.001020 -0.006129 -0.003768  0.023384  0.011089 -0.014955 -0.009447 -0.006131 -0.006511
1 2023-08-03 00:00:00+00:00        LTCUSDT    AVAXUSDT     -3 -0.003213  0.002417  0.011254 -0.011924  0.021722 -0.004724 -0.011867 -0.004804 -0.004023 -0.008078
2 2023-08-03 00:00:00+00:00        LTCUSDT     BNBUSDT     -3  0.001663  0.007884  0.000412 -0.004938  0.014475 -0.006115 -0.010254 -0.005387  0.000417  0.000416
3 2023-08-03 00:00:00+00:00        LTCUSDT     BTCUSDT     -3 -0.002569 -0.001855  0.000888  0.003721  0.019690 -0.006716 -0.004086 -0.000918 -0.000073 -0.003871
4 2023-08-03 00:00:00+00:00        LTCUSDT    DOGEUSDT     -3 -0.001764  0.028688 -0.017182 -0.011565  0.020952  0.004797  0.005570 -0.001846  0.013742 -0.024374
saved: ..\output\004_strategy\event_response_expanded.pkl
shape: (2320, 14)
```

12. Mean Response by Sigma Level and Lag

Computes the average forward return at each lag position, grouped by the reference coin's sigma level. This surfaces the aggregate directional tendency of follower assets following shocks of different magnitudes and signs.

In [12]:
```
lag_cols = [f"lag_{i+1}" for i in range(10)]

sigma_response = (
    event_response_expanded
    .groupby("sigma")[lag_cols]
    .mean()
    .sort_index()
)

print(sigma_response)
```

```
         lag_1     lag_2     lag_3     lag_4     lag_5     lag_6     lag_7     lag_8     lag_9    lag_10
sigma
-5     0.031235  0.005290  0.004586 -0.014319 -0.018990  0.012551 -0.006291  0.006079  0.009411 -0.002635
-4     0.023121  0.015307  0.008495 -0.006337 -0.010505  0.022606 -0.015286  0.013943  0.011418 -0.008052
-3     0.000910  0.004184 -0.000480  0.002353 -0.004133  0.013047 -0.007394  0.007178 -0.002873 -0.000021
 3     0.001176  0.015192 -0.001047  0.001384  0.005148 -0.004364 -0.004639 -0.003317  0.009706  0.005610
 4    -0.043455  0.012152  0.020587 -0.029966 -0.013297  0.039237 -0.058328 -0.025557  0.024736  0.023787
```

13. Focus Analysis: Negative 3-Sigma Events

Isolates events where the reference coin experienced a negative 3-sigma shock (sigma = −3) and examines the mean forward return profile across lags. Negative shocks are the focus because the observed propagation signal is strongest in this regime.

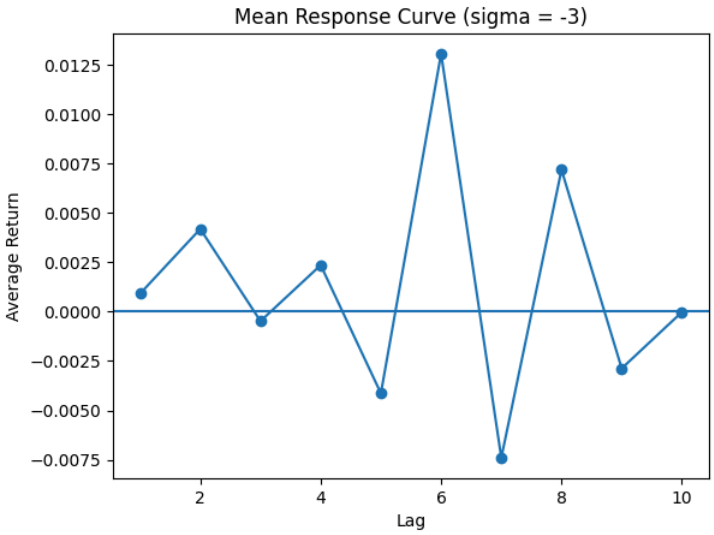
In [13]:
```
sigma3 = event_response_expanded[
    event_response_expanded["sigma"] == -3
]

sigma3_mean = sigma3[lag_cols].mean()

print(sigma3_mean)
```

```
lag_1     0.000910
lag_2     0.004184
lag_3    -0.000480
lag_4     0.002353
lag_5    -0.004133
lag_6     0.013047
lag_7    -0.007394
lag_8     0.007178
lag_9    -0.002873
lag_10   -0.000021
dtype: float64
```

14. Visualize Mean Response Curve (sigma = −3)

Plots the average forward return at each lag for sigma = −3 events. A persistent departure from zero across lags is evidence of a predictable directional tendency in follower assets.

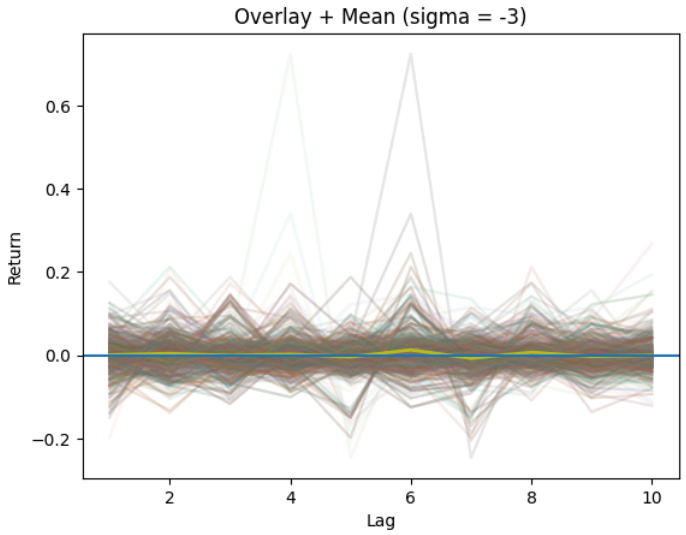
In [14]:
```
lags = list(range(1, 11))
values = sigma3_mean.values

plt.figure()
plt.plot(lags, values, marker='o')
plt.axhline(0)

plt.title("Mean Response Curve (sigma = -3)")
plt.xlabel("Lag")
plt.ylabel("Average Return")

plt.show()
```



15. Overlay Individual Event Paths

Plots each individual forward return path alongside the mean curve for sigma = −3 events. The overlay reveals the degree of dispersion around the mean -- a tight cluster supports signal reliability; high dispersion suggests noise dominates.

In [15]:
```
lags = list(range(1, 11))

# use RAW dataframe (has returns_vector)
sigma3_raw = event_response_df[
    event_response_df["sigma"] == -3
]

plt.figure()

# overlay all paths
for vec in sigma3_raw["returns_vector"]:
    plt.plot(lags, vec, alpha=0.05)

# mean line (from expanded dataframe)
plt.plot(lags, sigma3_mean.values, linewidth=2)

plt.axhline(0)

plt.title("Overlay + Mean (sigma = -3)")
plt.xlabel("Lag")
plt.ylabel("Return")

plt.show()
```



16. Dispersion Analysis by Lag

Computes mean, standard deviation, and signal-to-noise ratio (mean / std) at each lag for sigma = −3 events. Lags with the highest SNR are the best candidates for trade entry timing.

In [16]:
```
lag_cols = [f"lag_{i+1}" for i in range(10)]

dispersion_df = pd.DataFrame({
    "mean": sigma3[lag_cols].mean(),
    "std": sigma3[lag_cols].std(),
})

# signal-to-noise proxy
dispersion_df["abs_mean"] = dispersion_df["mean"].abs()
dispersion_df["snr"] = dispersion_df["abs_mean"] / dispersion_df["std"]

print(dispersion_df.sort_index())
```

```
           mean       std  abs_mean       snr
lag_1   0.000910  0.047036  0.000910  0.019350
lag_10 -0.000021  0.036198  0.000021  0.000568
lag_2   0.004184  0.045565  0.004184  0.091832
lag_3  -0.000480  0.045702  0.000480  0.010508
lag_4   0.002353  0.045893  0.002353  0.051278
lag_5  -0.004133  0.041092  0.004133  0.100583
lag_6   0.013047  0.066076  0.013047  0.197460
lag_7  -0.007394  0.045417  0.007394  0.162793
lag_8   0.007178  0.038473  0.007178  0.186569
lag_9  -0.002873  0.036251  0.002873  0.079251
```

17. Conditional Lag Transition Test

Tests whether lag_5 return direction is predictive of lag_6 direction. If follower assets that are still declining at lag_5 tend to continue declining at lag_6, this conditional structure can be exploited to improve trade timing.

In [17]:
```
subset = sigma3.copy()

# condition: lag_5 negative
subset = subset[subset["lag_5"] < 0]

print("count:", len(subset))

print("\nlag_6 stats (given lag_5 < 0):")
print("mean:", subset["lag_6"].mean())
print("median:", subset["lag_6"].median())
print("positive_ratio:", (subset["lag_6"] > 0).mean())
```

```
count: 825

lag_6 stats (given lag_5 < 0):
mean: 0.01433057008582787
median: 0.008432283600068757
positive_ratio: 0.5927272727272728
```

18. Refined Conditional Signal Test

Extends the transition test by applying two conditions -- `lag_5 < 0` (weak) and `lag_5 < −1%` (strong) -- and measuring the distributional properties of lag_6 returns under each. The stronger condition should produce a cleaner directional signal.

In [18]:
```
# base filter: sigma = -3 only
subset = event_response_expanded[
    event_response_expanded["sigma"] == -3
].copy()

print("base count (sigma = -3):", len(subset))

### condition 1: lag_5 < 0
subset_weak = subset[subset["lag_5"] < 0]

print("\n--- Condition: lag_5 < 0 ---")
print("count:", len(subset_weak))
print("mean:", subset_weak["lag_6"].mean())
print("median:", subset_weak["lag_6"].median())
```

```
print("positive_ratio:", (subset_weak["lag_6"] > 0).mean())

### condition 2: lag_5 < -1% (stronger signal)
subset_strong = subset[subset["lag_5"] < -0.01]

print("\n--- Condition: lag_5 < -1% ---")
print("count:", len(subset_strong))
print("mean:", subset_strong["lag_6"].mean())
print("median:", subset_strong["lag_6"].median())
print("positive_ratio:", (subset_strong["lag_6"] > 0).mean())
```

```
base count (sigma = -3): 1488

--- Condition: lag_5 < 0 ---
count: 825
mean: 0.01433057008582787
median: 0.008432283600068757
positive_ratio: 0.5927272727272728

--- Condition: lag_5 < -1% ---
count: 555
mean: 0.017522842507533224
median: 0.00869072359720402
positive_ratio: 0.5837837837837838
```

19. Leader-Follower Conditioned Signal Validation

Runs a direct comparison between the baseline (sigma = −3 only) and the conditioned signal (sigma = −3 and lag_5 < −1%). Summarizes mean, median, and win rate at lag_1, lag_2, and lag_3 for both groups.

This is the core signal validation test: does conditioning on a momentum continuation at lag_5 improve the predictability of returns at lag_2?

In [19]:
```
### build base dataset
base = event_response_expanded.copy()

print("total rows:", len(base))

### condition filter (leader condition)
conditioned = base[
    (base["sigma"] == -3) &
    (base["lag_5"] < -0.01)
].copy()

print("\nconditioned rows:", len(conditioned))

### baseline (for comparison)
baseline = base[
    base["sigma"] == -3
].copy()

print("baseline rows:", len(baseline))

### function to compute stats
def summarize(df, label):
    print(f"\n--- {label} ---")

    for lag in ["lag_1", "lag_2", "lag_3"]:
        mean = df[lag].mean()
        median = df[lag].median()
        pos = (df[lag] > 0).mean()

        print(f"{lag}:")
        print(f"  mean:   {mean:.5f}")
        print(f"  median: {median:.5f}")
        print(f"  pos%:   {pos:.3f}")

### run comparison
summarize(baseline, "BASELINE (sigma = -3)")
summarize(conditioned, "CONDITIONED (sigma=-3 & lag_5<-1%)")
```

```
total rows: 2320

conditioned rows: 555
baseline rows: 1488

--- BASELINE (sigma = -3) ---
lag_1:
  mean:   0.00091
  median: 0.00211
  pos%:   0.530
lag_2:
  mean:   0.00418
  median: 0.00498
  pos%:   0.538
lag_3:
  mean:   -0.00048
  median: -0.00317
  pos%:   0.467

--- CONDITIONED (sigma=-3 & lag_5<-1%) ---
lag_1:
  mean:   -0.00986
  median: -0.00790
  pos%:   0.420
lag_2:
  mean:   0.01279
  median: 0.01420
  pos%:   0.640
lag_3:
  mean:   -0.00158
  median: 0.00138
  pos%:   0.526
```

20. Final Signal Validation at lag_2

Full distributional analysis of the conditioned signal's lag_2 returns. Computes mean, median, standard deviation, win rate, average win, average loss, win/loss ratio, tail quantiles, and expected value.

A positive expected value with a win rate above 50% and a favorable win/loss ratio constitutes evidence of a tradable edge.

In [20]:
```
subset = event_response_expanded[
    (event_response_expanded["sigma"] == -3) &
    (event_response_expanded["lag_5"] < -0.01)
].copy()

lag = subset["lag_2"]

print("===== SAMPLE =====")
print("count:", len(lag))

print("\n===== CORE STATS =====")
print("mean:", lag.mean())
print("median:", lag.median())
print("std:", lag.std())

print("\n===== WIN / LOSS =====")
win_rate = (lag > 0).mean()
avg_win = lag[lag > 0].mean()
avg_loss = lag[lag <= 0].mean()

print("win_rate:", win_rate)
print("avg_win:", avg_win)
print("avg_loss:", avg_loss)
print("win/loss ratio:", avg_win / abs(avg_loss) if avg_loss != 0 else None)

print("\n===== TAIL RISK =====")
q = lag.quantile([0.01, 0.05, 0.1, 0.9, 0.95, 0.99])
print(q)

print("\n===== EXPECTED VALUE CHECK =====")
expected_value = win_rate * avg_win + (1 - win_rate) * avg_loss
print("expected_value:", expected_value)
```

```
===== SAMPLE =====
count: 555

===== CORE STATS =====
mean: 0.012791394930817896
median: 0.014204037877434406
std: 0.04262976120394329

===== WIN / LOSS =====
win_rate: 0.6396396396396397
avg_win: 0.036498532149250924
avg_loss: -0.029288773631900727
win/loss ratio: 1.2461611608584895

===== TAIL RISK =====
0.01   -0.096753
0.05   -0.064860
0.10   -0.041753
0.90    0.064272
0.95    0.088804
0.99    0.118180
Name: lag_2, dtype: float64

===== EXPECTED VALUE CHECK =====
expected_value: 0.0127913949308179
```

21. Trade Mapping

Converts the conditioned signal into a structured trade representation. For sigma = −3 events:

- If `lag_5 < −1%` (conditioned): use the lag_2 return as the trade return
- Otherwise (baseline): use the lag_1 return

Only rows with a non-zero signal return are retained as actual trade candidates.

In [21]:
```
df = event_response_expanded.copy()

### signal condition
df["use_lag2"] = (
    (df["sigma"] == -3) &
    (df["lag_5"] < -0.01)
)

### choose return based on rule
df["signal_return"] = 0.0

# conditioned -> lag_2
df.loc[df["use_lag2"], "signal_return"] = df.loc[df["use_lag2"], "lag_2"]

# default -> lag_1 (only for sigma = -3 events)
default_mask = (df["sigma"] == -3) & (~df["use_lag2"])
df.loc[default_mask, "signal_return"] = df.loc[default_mask, "lag_1"]

### keep only actual trades
trade_df = df[df["signal_return"] != 0].copy()

print("total trades:", len(trade_df))
print("lag2 trades:", trade_df["use_lag2"].sum())
print("lag1 trades:", (~trade_df["use_lag2"]).sum())
```

```
TRADE_FILE = STRATEGY_DIR / "trade_df.pkl"
trade_df.to_pickle(TRADE_FILE)
manifest[stage_label]["trade_df"] = str(TRADE_FILE)
print("saved:", TRADE_FILE)
print("shape:", trade_df.shape)
```

```
total trades: 1486
lag2 trades: 555
lag1 trades: 931
saved: ..\output\004_strategy\trade_df.pkl
shape: (1486, 16)
```

22. Build the PnL Time Series

Aggregates trade returns by date (equal-weighted across all active trades on a given day) to produce a daily return series. Computes total return, annualized volatility, Sharpe ratio, and maximum drawdown as a preliminary performance check.

Note: this is a gross (pre-cost) estimate; transaction costs are applied in the next section.

In [22]:
```
### aggregate by date (equal weight)
daily_returns = (
    trade_df
    .groupby("event_date")["signal_return"]
    .mean()
    .sort_index()
)

print("days:", len(daily_returns))
print("mean daily return:", daily_returns.mean())
print("std daily return:", daily_returns.std())

### equity curve
equity_curve = (1 + daily_returns).cumprod()

### basic stats
total_return = equity_curve.iloc[-1] - 1
volatility = daily_returns.std() * np.sqrt(trading_days)

sharpe = (
    daily_returns.mean() /
    daily_returns.std()
) * np.sqrt(trading_days)

drawdown = equity_curve / equity_curve.cummax() - 1
max_drawdown = drawdown.min()

print("\n===== PERFORMANCE =====")
print("total_return:", total_return)
print("volatility:", volatility)
print("sharpe:", sharpe)
print("max_drawdown:", max_drawdown)

DAILY_RETURNS_FILE = STRATEGY_DIR / "daily_returns.pkl"
daily_returns.to_pickle(DAILY_RETURNS_FILE)
manifest[stage_label]["daily_returns"] = str(DAILY_RETURNS_FILE)
print("saved:", DAILY_RETURNS_FILE)
print("shape:", daily_returns.shape)
```

```
days: 66
mean daily return: 0.00391723395634095
std daily return: 0.039743430243978216

===== PERFORMANCE =====
total_return: 0.22826441629494654
volatility: 0.6309073960452568
sharpe: 1.5646400140268903
max_drawdown: -0.20238612360361818
saved: ..\output\004_strategy\daily_returns.pkl
shape: (66,)
```

23. Apply Transaction Costs

Subtracts a per-trade cost at three levels (20, 30, and 40 basis points) and recomputes all performance metrics. This provides a preliminary sensitivity check on the strategy's robustness to realistic execution friction before formal backtesting in Stage 5.

In [23]:
```
cost_levels = {
    "20bps": 0.002,
    "30bps": 0.003,
    "40bps": 0.004,
}

results = {}

for label, cost in cost_levels.items():

    df_cost = trade_df.copy()

    # subtract cost per trade
    df_cost["net_return"] = df_cost["signal_return"] - cost

    # aggregate daily
    daily = (
        df_cost
        .groupby("event_date")["net_return"]
        .mean()
        .sort_index()
    )

    # equity
    equity = (1 + daily).cumprod()
```

```
    # stats
    total_return = equity.iloc[-1] - 1
    vol = daily.std() * np.sqrt(trading_days)

    sharpe = (
        daily.mean() /
        daily.std()
    ) * np.sqrt(trading_days)

    dd = equity / equity.cummax() - 1
    max_dd = dd.min()

    results[label] = {
        "total_return": total_return,
        "sharpe": sharpe,
        "max_drawdown": max_dd,
    }

# print results
for k, v in results.items():
    print(f"\n===== {k} =====")
    for metric, val in v.items():
        print(f"{metric}: {val}")
```

```
===== 20bps =====
total_return: 0.0765623528053303
sharpe: 0.7657906057631026
max_drawdown: -0.20755157649101985

===== 30bps =====
total_return: 0.007789509626073388
sharpe: 0.3663659016312094
max_drawdown: -0.21901945559788294

===== 40bps =====
total_return: -0.05665245170947064
sharpe: -0.03305880250068437
max_drawdown: -0.23545836215610827
```

24. Isolate High-Quality Trade Subset

Retains only trades generated by the conditioned signal (`use_lag2 = True`) -- the highest-conviction subset. Trades from the default lag_1 path are excluded.

This reduced set is the primary input to the Stage 5 backtest.

In [24]:
```
### keep only strongest condition
reduced_df = trade_df[trade_df["use_lag2"]].copy()

print("original trades:", len(trade_df))
print("reduced trades:", len(reduced_df))
print("reduction %:", 1 - len(reduced_df)/len(trade_df))

REDUCED_DF_FILE = STRATEGY_DIR / "reduced_df.pkl"
reduced_df.to_pickle(REDUCED_DF_FILE)
manifest[stage_label]["reduced_df"] = str(REDUCED_DF_FILE)
print("saved:", REDUCED_DF_FILE)
print("shape:", reduced_df.shape)
```

```
original trades: 1486
reduced trades: 555
reduction %: 0.626514131897712
saved: ..\output\004_strategy\reduced_df.pkl
shape: (555, 16)
```

25. PnL for Reduced (Conditioned) Trades

Recomputes the performance summary using only the high-quality conditioned trade subset. This is the baseline performance figure before the stress-grid evaluation in Stage 5.

In [25]:
```
daily_returns = (
    reduced_df
    .groupby("event_date")["signal_return"]
    .mean()
    .sort_index()
)

equity_curve = (1 + daily_returns).cumprod()

total_return = equity_curve.iloc[-1] - 1
volatility = daily_returns.std() * np.sqrt(trading_days)

sharpe = (
    daily_returns.mean() /
    daily_returns.std()
) * np.sqrt(trading_days)

drawdown = equity_curve / equity_curve.cummax() - 1
max_drawdown = drawdown.min()

print("\n===== REDUCED PERFORMANCE =====")
print("total_return:", total_return)
print("sharpe:", sharpe)
print("max_drawdown:", max_drawdown)
```

```
===== REDUCED PERFORMANCE =====
total_return: 0.3657609561986037
sharpe: 3.6818776151931667
max_drawdown: -0.11981425265943557
```

26. Stress Test the Reduced Trade Set

Applies the same cost sensitivity analysis (20, 30, 40 bps) to the reduced conditioned trade set. A strategy that retains a positive Sharpe ratio across all cost levels is a strong candidate for formal backtesting.

In [26]:
```
cost_levels = {
    "20bps": 0.002,
    "30bps": 0.003,
    "40bps": 0.004,
}

results = {}

for label, cost in cost_levels.items():

    df_cost = reduced_df.copy()

    # subtract cost per trade
    df_cost["net_return"] = df_cost["signal_return"] - cost

    # aggregate daily
    daily = (
        df_cost
        .groupby("event_date")["net_return"]
        .mean()
        .sort_index()
    )

    # equity
    equity = (1 + daily).cumprod()

    # stats
    total_return = equity.iloc[-1] - 1
    vol = daily.std() * np.sqrt(trading_days)

    sharpe = (
        daily.mean() /
        daily.std()
    ) * np.sqrt(trading_days)

    dd = equity / equity.cummax() - 1
    max_dd = dd.min()

    results[label] = {
        "total_return": total_return,
        "sharpe": sharpe,
        "max_drawdown": max_dd,
    }

# print results
for k, v in results.items():
    print(f"\n===== {k} =====")
    for metric, val in v.items():
        print(f"{metric}: {val}")
```

```
===== 20bps =====
total_return: 0.26137425580847573
sharpe: 2.8129825140458884
max_drawdown: -0.12880842980067475

===== 30bps =====
total_return: 0.21214001989379616
sharpe: 2.3785349634722497
max_drawdown: -0.13327786403218322

===== 40bps =====
total_return: 0.1647812371661166
sharpe: 1.9440874128986112
max_drawdown: -0.1377289377277774
```

999. Persist the Manifest

Records all artifact paths and a completion timestamp for this stage into the shared manifest.

In [27]:
```
manifest[stage_label]["timestamp"] = datetime.now(UTC).isoformat()
pd.to_pickle(manifest, MANIFEST_FILE)
print("manifest saved:", MANIFEST_FILE)
print("pipeline stages:", sorted(manifest.keys()))
print(f"artifacts in {stage_label}:", sorted(manifest[stage_label].keys()))
```

```
manifest saved: ..\output\manifest.pkl
pipeline stages: ['001_download', '002_enrich', '003_analysis', '003a_regimes', '004_strategy']
artifacts in 004_strategy: ['daily_returns', 'event_response_df', 'event_response_expanded', 'reduced_df', 'timestamp', 'trade_df']
```

In [ ]: